PlagiarismAI

REVIEW 2 IEEE PAPER.docx

Document similarity report

Document Details

Reference identification	Institute of Aeronautical Engineering (IARE)
Submission ID	reference-style-review2
Submission date	2026-11-10T07:47:18.428002+00:00
Report generated	2026-11-10T07:55:36+00:00
File Name	REVIEW 2 IEEE PAPER.docx
File Size	2,652,357 bytes
Pages	Not available
Words	4,463
Characters	35,000

Integrity Overview

0.0% Overall Similarity

The combined total of valid unique matches in the authorized source corpus. Direct and near-duplicate matches are calculated separately; semantic similarity is not added to this percentage.

Filtered from the Report

Bibliography	Quoted Text	Cited Text	Small Matches
Not applied	Not applied	Not applied	Not applied

Match Groups

Group	Matches	Percentage
Not Cited or Quoted	Unavailable	0.0%
Missing Quotations	Unavailable	Unavailable
Missing Citation	Unavailable	Unavailable
Cited and Quoted	Unavailable	Unavailable

Top Sources

Category	Sources	Contribution
Internet sources	0	Unavailable
Publications	0	Unavailable
Submitted works	0	Unavailable
Uploaded documents	0	No matches

Integrity Flags

No suspicious text manipulations found.

Integrity Overview

Top Sources

The sources with the highest number of meaningful matches within the submission. Duplicate content hashes are represented once.

No matching sources were returned from the available authorized corpus.

Source Coverage

Category	Availability	Count
Internet sources	Unavailable	0
Publications	Unavailable	0
Submitted works	Unavailable	0
Uploaded documents	No matches	0

Matching Summary

0 unique source(s), 0 direct match(es), 0 near-duplicate match(es), and 0 grouped passage block(s).

Source colours used in the document view:

Source	Colour	Match type

REVIEW 2 IEEE PAPER.docx | Analysed document

StreamView: A Self-Hosted Streaming Materialized View Engine

Mrs .

K .

Nagamani Dept.

of Computer Science and Engineering

MLR Institute of Technology Hyderabad, India.

astakalanagamani78@gmail.com

P.

Srinath

Dept.

of Computer Science and Engineering

MLR Institute of Technology

Hyderabad, India.

23r21a05y0@mlrit.ac.in

Assis Mohanty

Dept.

of Computer Science and Engineering

MLR Institute of Technology Hyderabad, India.

23r21a05t0@mlrit.ac.in

V.

Shiva Ganesh

Dept.

of Computer Science and Engineering

MLR Institute of Technology Hyderabad, India.

24r21a0532@mlrit.ac.in

ABSTRACT

Real-time analytics over continuously generated data requires efficient event processing and timely access to aggregated results.

StreamView is a self-hosted streaming materialized-view engine designed to reduce dependence on managed cloud services.

It combines message-based event ingestion, stream processing, and persistent state management.

The processing pipeline supports filtering, transformation, aggregation, and event-time operations.

An LSM-tree-based state store manages persistent processing state.

Write-ahead logging supports durability and recovery after system failures.

Watermarks help manage late-arriving events according to configured policies.

Incremental view maintenance updates affected results as new events arrive.

This approach aims to avoid repeated computation over historical data.

The architecture is intended to support low-latency analytical queries and continuous data processing.

A web interface can help users configure streams, manage materialized views, and monitor system activity.

StreamView also provides a self-hosted alternative that gives developers greater control over deployment and resources.

Its performance can be evaluated through throughput, query latency, view-update latency, recovery behavior, and resource utilization.

The project provides a foundation for studying practical trade-offs in self-hosted real-time analytics infrastructure.

Keywords: Stream Processing, Materialized Views, Incremental Computation, Real-Time Analytics, Event Streaming, Message Broker, LSM-Tree, Write-Ahead Logging, Event-Time Processing, Self-Hosted Infrastructure, Query Optimization, Distributed Systems.

INTRODUCTION

Modern applications continuously generate large volumes of data that require efficient processing and timely analysis.

However, traditional batch-processing approaches can delay analytical results, while repeated query execution increases computational overhead.

These challenges highlight the need for systems that can process incoming events continuously and maintain updated analytical results with minimal redundant computation.

Distributed systems and incremental processing techniques provide opportunities to improve data processing efficiency and query performance [1], [2], [3].

A streaming materialized-view engine can maintain precomputed query results as new events arrive.

Unlike conventional approaches that repeatedly process historical data, incremental view maintenance updates only the affected results, reducing unnecessary computation.

Event-time processing and watermark mechanisms help manage late-arriving events, while persistent state management supports reliable processing and recovery [4], [5], [6].

This paper proposes StreamView, a self-hosted streaming materialized-view engine integrating event ingestion, continuous stream processing, and incremental view maintenance.

The framework combines message-broker-based ingestion, filtering and aggregation operations, persistent state storage, and query interfaces to support real-time analytics.

The proposed architecture aims to improve query efficiency and provide greater control over deployment and infrastructure resources.

Its effectiveness can be evaluated using event-processing throughput, query latency, view-update latency, recovery behavior, and resource utilization under different workloads.

LITERATURE SURVEY

1.

Real-Time Stream Processing Systems (2024) – Continuous event processing enables applications to analyze incoming data with minimal delay.

However, maintaining consistent analytical results under high event volumes remains a challenge.

2.

Incremental Materialized View Maintenance (2023) – Incremental computation updates query results when source data changes, reducing repeated processing.

However, complex aggregations and large-scale workloads can increase update overhead.

3.

Distributed Stream Processing Architectures (2024) – Distributed processing frameworks improve scalability by dividing workloads across multiple workers.

However, coordination, network communication, and fault recovery can introduce additional overhead.

4.

Event-Time Processing in Streaming Applications (2023) – Event-time processing and watermarks help systems handle delayed and out-of-order events.

However, selecting appropriate watermark policies can affect result completeness and processing latency.

5.

Persistent State Management for Stream Processing (2024) – Persistent state stores enable streaming applications to retain intermediate results and recover after failures.

However, storage performance and recovery mechanisms can influence overall system efficiency.

6.

Write-Ahead Logging for Reliable Data Systems (2023) – Write-ahead logging improves durability by recording changes before applying them to persistent state.

However, log management and recovery procedures can increase storage and processing overhead.

7.

Embedding Models for Technical Information Retrieval (2023) – Embedding-based search helps developers retrieve relevant documentation even when search terms differ from the original wording.

However, continuous index updates can increase memory consumption and computational costs.

8.

Third-Party Integrations in Collaboration Platforms (2022) – Integrating services such as GitHub, Jira, and Slack reduces application switching.

However, maintaining reliable integrations requires ongoing API updates and webhook management.

9.

AI-Based Task Recommendations for Software Teams (2024) – Machine-learning models can recommend task assignments using historical work patterns.

However, recommendations may become outdated when priorities and team workloads change.

10.

Semantic Search for Developer Communication (2024) – Vector-based retrieval helps developers locate previous discussions and technical decisions.

However, combining semantic search with live messaging can increase server memory usage under heavy workloads.

11.

Enterprise Knowledge Management Using AI (2024) – AI-assisted knowledge management can improve access to technical information and automate routine workflows.

However, integrating documentation with daily development activities remains challenging.

12.

Cloud Collaboration for Distributed Engineering Teams (2023) – Cloud platforms simplify deployment and support remote collaboration.

However, organizations must address data privacy, access control, and data-residency requirements.

13.

Conversational AI for Software Development (2024) – AI chatbots can answer common development questions and support onboarding.

However, complex debugging tasks involving multiple codebases may require additional contextual information.

14.

Vector Databases for Enterprise Search (2024) – Vector databases support semantic retrieval by matching contextual meaning rather than exact keywords.

However, high query volumes can increase memory consumption and affect write performance.

15.

Machine Learning for Software Development Workflows (2023) – Machine-learning techniques can assist with bug classification, issue prioritization, and workflow automation.

However, their effectiveness depends on the availability of accurate historical data.

16.

Recommendation Systems for Team Productivity (2024) – Historical commits and task records can help identify developers suited to particular assignments.

However, limited historical activity can reduce recommendation accuracy for new team members.

17.

Large Language Models for Context-Aware Collaboration (2024) – Large language models can summarize meetings and technical discussions.

However, generated summaries may contain inaccuracies when project context or code versions are outdated.

18.

AI-Enabled Enterprise Collaboration Platforms (2024) – Combining communication, task management, and search in one platform improves information accessibility.

However, integrating different permission models and data structures introduces administrative complexity.

19.

Semantic Search for Software Engineering Documentation (2024) – Vector search helps developers discover relevant technical documents using natural-language queries.

However, retrieving information distributed across issue trackers, pull requests, and discussions remains challenging.

20.

Cross-Platform Knowledge Discovery for Engineering Teams (2024) – Indexing pull requests, chat messages, and technical

documentation can improve cross-platform knowledge discovery.

However, maintaining contextual relevance requires regular indexing updates and efficient management of computational resources.

Important: These are rewritten literature-review summaries based on the descriptions you provided, not verified publications.

Before submitting your IEEE paper, verify each paper's actual title, authors, publication venue, year, and findings.

Do not present unverified titles or numerical claims as published research.

GAPS IDENTIFIED

Fragmented Data Processing

Traditional data-processing systems often rely on separate tools for event ingestion, processing, storage, and querying, making real-time analytics difficult to manage efficiently.

Limited Incremental Computation

Repeatedly recomputing query results can increase processing overhead.

Efficient incremental maintenance of materialized views remains an important challenge.

Inefficient Query Execution

Traditional query execution may require processing large amounts of historical data, increasing response times for continuously changing datasets.

Complex Infrastructure Management

Existing streaming platforms may require complex configuration, deployment, and maintenance, creating challenges for developers and smaller organizations.

Limited Processing Visibility

Monitoring event ingestion, processing performance, materialized views, and query execution across separate tools can make system management more difficult.

Challenges in State Recovery

System failures can interrupt stream processing and affect stored results.

Reliable state persistence and recovery mechanisms are necessary to maintain processing continuity.

Resource Utilization and Scalability

Increasing event volumes can raise CPU, memory, and storage requirements.

Efficient resource utilization and scalable processing are essential for maintaining performance.

Handling High-Volume Event Streams

Processing a continuously increasing volume of events while maintaining low latency is challenging.

Efficient ingestion and processing mechanisms are required to handle high-throughput workloads.

Between Events and Materialized Views

Delayed, duplicated, or out-of-order events can cause discrepancies in analytical results.

Reliable update mechanisms are needed to maintain accurate materialized views.

Limited Benchmarking of Self-Hosted Solutions

The performance of self-hosted streaming materialized-view engines may vary across workloads and hardware configurations.

Systematic evaluation of throughput, query latency, recovery time, and resource consumption is needed to understand their practical limitations.

PROBLEM STATEMENT

Modern applications continuously generate large volumes of data that require real-time processing and efficient analytical access.

Traditional batch-processing systems often introduce delays, while repeatedly executing queries over large datasets increases computational overhead.

These limitations make it challenging to obtain up-to-date analytical results as new events arrive.

Additionally, managing separate tools for event ingestion, stream processing, state storage, and querying can increase infrastructure complexity and operational costs.

StreamView addresses these challenges by providing a self-hosted streaming materialized-view engine that integrates continuous event ingestion, stream processing, and incremental view maintenance into a unified platform.

It processes incoming events through filtering, transformation, and aggregation operations while updating materialized views as relevant data changes.

Instead of recomputing query results over the entire dataset, the system aims to update only the affected results, reducing redundant computation.

By providing query interfaces and a monitoring dashboard, StreamView aims to simplify system management, improve analytical query efficiency, and give developers greater control over deployment and resource utilization.

MOTIVATION

Modern applications require continuous data processing to support timely decision-making and real-time analytics.

However, traditional data-processing systems often depend on batch execution or repeated query computation, which can increase response times and resource consumption.

As data volumes grow, maintaining up-to-date analytical results efficiently becomes increasingly important.

These challenges motivate the development of a unified streaming materialized-view engine that simplifies event processing and reduces redundant computation.

StreamView is motivated by the need for a self-hosted platform that combines event ingestion, stream processing, persistent state management, and incremental materialized-view maintenance.

By processing incoming events continuously and updating only the affected view results, the system aims to improve query efficiency and reduce unnecessary processing overhead.

Its monitoring dashboard and query interfaces provide visibility into stream activity, view updates, and system performance.

Furthermore, a self-hosted deployment gives developers greater control over infrastructure, resource allocation, and operational costs.

StreamView therefore provides a foundation for exploring efficient, manageable, and cost-conscious real-time analytics without relying entirely on managed cloud services.

RESEARCH OBJECTIVES

Develop a Self-Hosted Streaming Engine

Design and develop StreamView as a self-hosted streaming materialized-view engine that integrates event ingestion, stream processing, persistent state management, and analytical querying.

The system aims to simplify real-time data processing through a unified architecture.

Implement Continuous Stream Processing

Enable continuous ingestion and processing of incoming events through operations such as filtering, transformation, and aggregation.

The objective is to process data as it arrives and reduce the delays associated with traditional batch-processing approaches.

Achieve Incremental Materialized View Maintenance

Develop mechanisms to maintain materialized views by updating affected query results whenever new events arrive.

This approach aims to minimize redundant computation and improve the efficiency of analytical queries over continuously changing datasets.

Improve State Persistence and Recovery

Explore persistent state storage and recovery mechanisms to preserve processing state and restore operations after failures.

Where implemented, techniques such as write-ahead logging and event-time watermarks can support durability and the handling of late-arriving events.

Traditional data-processing systems often depend on scheduled jobs and repeated queries to generate analytical results.

As data volumes increase, these approaches may lead to slower responses, unnecessary computation, and higher resource consumption.

Managing separate processing and storage components can also increase system complexity, making continuous data analysis more difficult.

Periodic refreshes may prevent users from accessing the latest information immediately.

Reprocessing entire datasets for simple updates can waste computational resources and increase query execution time.

These limitations highlight the need for a unified system that processes incoming events continuously and maintains analytical results efficiently.

StreamView aims to address these challenges through stream processing and incremental materialized-view maintenance.

EXSISTING SYSTEM

FIG 7.1 Existing System

The modern software team uses all sorts of different tools for its work: chats are in Slack, tasks – in Jira, documentation – in Notion, code reviews – in GitHub.

Every tool is great on its own, but they all require a separate set of notifications that clutter up the user's time and confuse the context.

PROPOSED METHODOLOGY

StreamView is designed as a self-hosted streaming materialized-view engine that integrates continuous event ingestion, stream processing, persistent state management, and analytical querying within a unified platform.

Incoming events are delivered through a message broker such as Redpanda, which provides Kafka-compatible event ingestion.

The processing engine consumes these events and performs operations such as filtering, transformation, and aggregation to generate continuously updated results

.

The backend manages stream configurations, materialized-view definitions, query requests, and communication between system components.

A persistent state store maintains the data required for processing and view updates.

Where implemented, write-ahead logging (WAL) can support recovery after failures, while event-time processing and watermarks can help manage late-arriving events.

Materialized views are updated incrementally as new events arrive, reducing the need to repeatedly recompute analytical results over the complete dataset.

The web dashboard provides an interface for creating and managing streams, configuring materialized views, executing queries, and monitoring system activity.

Backend APIs connect the dashboard with the processing engine and state store.

REST APIs support configuration and query operations, while WebSocket connections can provide real-time updates about stream activity and processing status.

Authentication, authorization, and organization-level access controls can be incorporated to protect configurations and analytical data.

The system can be evaluated using event throughput, query latency, view-update latency, recovery behavior, and resource utilization.

The following steps should be taken while using StreamView:

Step 1: Access the Dashboard – Open the StreamView web application and access the dashboard to view stream activity, materialized views, query information, and system performance.

Step 2: Configure an Event Stream – Create or select a stream and configure its required properties, including the event source, topic, and relevant processing settings.

Step 3: Ingest Events – Send events from a producer application to the configured message broker.

StreamView consumes the incoming events for continuous processing.

Step 4: Configure Stream Processing – Define the required operations, such as filtering, transformation, and aggregation, to process events according to the analytical requirements.

Step 5: Create a Materialized View – Define a view based on the required query or aggregation.

The engine processes incoming events and maintains the corresponding results.

Step 6: Query Materialized Results – Execute queries through the query interface or API to retrieve the latest available results without repeatedly processing the entire underlying dataset.

Step 7: Monitor Performance and Processing – Use the dashboard to observe stream activity, view updates, query latency, event throughput, and available resource metrics.

Step 8: Manage Persistence and Recovery – Monitor stored processing state and use the recovery mechanisms supported by the implementation to restore operations following a failure.

Step 9: Evaluate System Performance – Test the engine under different event volumes and query workloads, recording throughput, query latency, view-update latency, and resource consumption to assess its effectiveness.

Note: Keep claims about WAL, watermarks, WebSockets, authentication, and recovery aligned with the features actually implemented in your StreamView project.

FIG 8.1 Proposed workflow Diagram

Step 10: Real-Time Monitoring – StreamView monitors event ingestion, stream-processing activity, and materialized-view updates through the dashboard.

Users can observe system performance and identify processing delays or potential bottlenecks.

Step 11: Update Stream and View Configurations – Users can modify stream settings, processing operations, and materialized-view definitions according to changing analytical requirements.

The system applies the updated configurations based on its supported configuration and execution mechanisms.

Step 12: Performance Analysis and Optimization – Evaluate StreamView under different workloads by measuring event throughput, query latency, view-update latency, CPU usage, and memory consumption.

These measurements help identify performance bottlenecks and guide future improvements to processing efficiency and resource utilization.

ALGORITHM

The following steps describe the processing workflow of the StreamView application.

Algorithm 1: Streaming Event Ingestion and Incremental Materialized View Maintenance Pipeline

Input: Incoming Events \(E\), Stream Configuration \(C\), Materialized View Definitions \(V\)

Output: Updated Materialized Views \(V'\), Query Results \(R\)

1.

Initialize the message broker, stream-processing engine, persistent state store, and query interface.

2.

Configure the event stream and subscribe to the required message topic.

3.

For each incoming event \(e\) in \(E\), do:

4.

Receive event \(e\) from the message broker.

5.

Validate the event structure and required fields.

6.

Apply the configured filtering and transformation operations to \(e\).

7.

If the event satisfies the processing conditions, compute the required aggregation or state changes.

8.

Identify the materialized views affected by the processed event.

9.

Incrementally update the affected view results using the new event and existing state.

10.

Persist the updated state and view data using the supported storage mechanism.

11.

Continue processing incoming events while updating the relevant materialized views.

12.

On receiving a query \(Q\) from the client interface:

13.

Identify the corresponding materialized view and retrieve the required results from the state store.

14.

Return the query results \(R\) to the client interface.

15.

Monitor processing activity, query latency, event throughput, and resource utilization.

16.

Repeat the process until the stream is stopped or the system is shut down.

Note: This algorithm describes the intended StreamView workflow.

Ensure that validation, persistence, incremental updates, and monitoring steps match your actual implementation.

SYSTEM OVERVIEW

StreamView is a self-hosted streaming materialized-view engine designed to process continuously generated events and provide efficient access to analytical results.

The system enables users to configure event streams, define processing operations, create materialized views, and execute queries through a unified web interface.

By integrating event ingestion, stream processing, persistent state management, and querying, StreamView aims to simplify real-time analytics while reducing unnecessary computation and dependence on managed cloud services.

From an architectural perspective, StreamView uses a message broker such as Redpanda for event ingestion, a stream-processing engine for filtering, transformation, and aggregation, and a persistent state store for maintaining processing data and materialized-view results.

The engine updates affected views as new events arrive, allowing queries to retrieve precomputed results rather than repeatedly processing the entire dataset.

Backend APIs connect these components with the web dashboard, enabling stream configuration, view management, query execution, and performance monitoring.

The system is intended to support real-time analytical workloads while providing greater control over deployment and infrastructure resources.

Its effectiveness can be evaluated through event-processing throughput, query latency, view-update latency, recovery behavior, and CPU and memory utilization.

This architecture provides a foundation for developing and evaluating a practical, self-hosted solution for continuous data processing and incremental analytics.

The system maintains precomputed query results and updates affected data when new events arrive, reducing the need to repeatedly process the entire dataset.

Its storage and query components work together to make analytical results available through backend APIs.

Performance can be assessed using event throughput, query latency, view-update latency, and resource utilization.

By combining continuous event processing with incremental view maintenance, StreamView aims to improve query efficiency, simplify data management, and provide greater control over deployment and infrastructure costs.

StreamView is a self-hosted platform designed to process continuous data streams and maintain updated analytical results.

It uses a message broker to receive events and a processing engine to perform filtering, transformation, and aggregation.

Materialized views are updated incrementally as new events arrive, reducing repeated computation.

The dashboard enables users to manage streams, execute queries, and monitor system performance.

SYSTEM ARCHITECTURE

The StreamView system architecture consists of five major layers: the client interface, application backend, event ingestion and stream processing, persistent storage, and query and monitoring.

The client interface provides a web dashboard for managing streams, configuring materialized views, executing queries, and monitoring system activity.

The application backend manages business logic, API requests, stream configurations, and communication between components.

The event ingestion layer uses a message broker such as Redpanda to receive continuous data streams, while the processing engine performs filtering, transformation, and aggregation.

The persistent storage layer maintains processing state and materialized-view results for efficient retrieval.

Finally, the query and monitoring layer enables users to access analytical results and observe system performance through metrics such as event throughput, query latency, view-update latency, and resource utilization.

These layers work together to support continuous data processing, incremental materialized-view maintenance, and efficient real-time analytics in a self-hosted environment.

FIG 11.1 System Architecture

RESULTS

The results of the StreamView system demonstrate the major functional components of the proposed streaming materialized-view engine.

The dashboard provides an overview of active streams, materialized views, event throughput, and system health.

The stream management module enables users to configure and monitor event streams, while the materialized-view module displays view definitions, execution status, and update information.

The query interface allows users to retrieve analytical results from materialized views using SQL-like queries.

These components work together to provide a unified environment for managing continuous data processing and accessing analytical information.

The monitoring and event ingestion modules provide additional visibility into system operations.

Incoming events are received through the message broker and processed by the stream-processing engine before the relevant materialized views are updated.

The monitoring interface can display metrics such as event throughput, query latency, view-update latency, and CPU and memory utilization.

These measurements help evaluate system performance and identify potential processing bottlenecks.

Overall, StreamView aims to reduce repeated computation and provide timely analytical results through incremental view maintenance.

Actual performance improvements should be established through experiments using measured results from the implemented system rather than illustrative dashboard values.

FIG 12.1: Result

By using Weaviate's semantic search and Google Gemini, this workspace is able to address the collaborative and operational shortcomings faced by teams when it comes to their project knowledge.

By directly integrating with GitHub, Notion and Google Meet, it is able to extract mentions, pull requests, notes, and documentation in a single pane to avoid the chaos of multiple tools that dilutes team focus on the work itself.

FUTURE ENHANCEMENTS

In the future, StreamView can be enhanced to support distributed stream processing and horizontal scalability to handle increasing event volumes and large-scale analytical workloads.

Advanced query optimization techniques and adaptive materialized-view maintenance can be introduced to improve processing efficiency and reduce query latency.

Additional features such as exactly-once processing, fault tolerance, automated recovery, and improved handling of late-arriving events can strengthen system reliability.

Integration with multiple data sources and message brokers can expand its flexibility, while advanced monitoring, alerting, and resource management can help users identify performance bottlenecks.

Furthermore, container orchestration using Docker and Kubernetes can simplify deployment and scaling across different environments.

Security enhancements, including role-based access control, encrypted communication, and audit logging, can improve data protection.

Finally, machine learning techniques could be explored to predict workloads, optimize resource allocation, and recommend materialized views based on query patterns, making StreamView a more scalable, efficient, and intelligent real-time analytics platform.

CONCLUSION

The proposed StreamView architecture integrates event ingestion, stream processing, incremental materialized-view maintenance, and analytical querying into a unified platform.

Instead of repeatedly processing entire datasets, the system continuously consumes events through a message broker such as Redpanda and updates relevant materialized views as new data arrives.

The stream-processing engine performs filtering, transformation, and aggregation, while persistent storage maintains processing state and view results for efficient retrieval.

Backend APIs connect these components to a web dashboard, enabling users to configure streams, manage materialized views, execute queries, and monitor system performance.

This architecture aims to reduce redundant computation, improve analytical query efficiency, and provide timely access to continuously updated results.

By supporting self-hosted deployment and centralized monitoring, StreamView gives developers greater control over infrastructure, resource utilization, and the management of real-time data-processing workloads.

REFERENCES

[1] M.

Fragkoulis, P.

Carbone, V.

Kalavri, and A.

Katsifodimos, “A survey on the evolution of stream processing systems,” The VLDB Journal, vol.

33, pp.

507–541, 2024.

[Online].

Available: https://doi.org/10.1007/s00778-023-00819-8

[2] Y.

Yang, L.

Golab, and M.

T.

Özsu, “ViewDF: Declarative incremental view maintenance for streaming data,” Information Systems, vol.

68, pp.

55–67, 2017.

[Online].

Available: https://doi.org/10.1016/j.is.2017.07.002

[3] M.

Budiu, T.

Chajed, F.

McSherry, L.

Ryzhyk, and V.

Tannen, “DBSP: Automatic incremental view maintenance for rich query languages,” Proceedings of the VLDB Endowment, vol.

16, no.

7, pp.

1601–1614, 2023.

[Online].

Available: https://doi.org/10.14778/3587136.3587137

[4] A.

Gupta and I.

S.

Mumick, “Maintenance of materialized views: Problems, techniques, and applications,” IEEE Data Engineering Bulletin, vol.

18, no.

2, pp.

3–18, 1995.

[5] A.

Gupta, I.

S.

Mumick, and V.

S.

Subrahmanian, “Maintaining views incrementally,” ACM SIGMOD Record, vol.

22, no.

2, pp.

157–166, 1993.

[Online].

Available: https://doi.org/10.1145/170036.170066

[6] K.

A.

Ross, D.

Srivastava, and S.

Sudarshan, “Materialized view maintenance and integrity constraint checking: Trading space for time,” in Proc.

ACM SIGMOD Int.

Conf.

Management of Data, 1996, pp.

447–458.

[7] S.

Ceri and J.

Widom, “Deriving production rules for incremental view maintenance,” in Proc.

17th Int.

Conf.

Very Large Data Bases (VLDB), 1991, pp.

577–589.

[8] J.

A.

Blakeley, P.-Å.

Larson, and F.

W.

Tompa, “Efficiently updating materialized views,” in Proc.

ACM SIGMOD Int.

Conf.

Management of Data, 1986, pp.

61–71.

[9] T.

Akidau et al., “The Dataflow Model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-, vol.

8, no.

12, pp.

1792–1803, 2015.

[10] T.

Akidau et al., “MillWheel: Fault-tolerant stream processing at internet scale,” in Proc.

39th Int.

Conf.

Very Large Data Bases (VLDB), 2013.

[11] P.

Carbone et al., “Apache Flink: Stream and batch processing in a single engine,” IEEE Data Engineering Bulletin, vol.

38, no.

4, pp.

28–38, 2015.

[12] M.

Zaharia et al., “Discretized streams: Fault-tolerant streaming computation at scale,” in Proc.

ACM Symp.

Operating Systems Principles (SOSP), 2013, pp.

423–438.

[13] T.

Akidau et al., “Watermarks in stream processing systems: Semantics Dataflow,” Proceedings of the VLDB Endowment, vol.

14, no.

12, pp.

3135–3147, 2021.

[Online].

Available: https://doi.org/10.14778/3476311.3476389

[14] M.

A.

Shah, J.

M.

Hellerstein, and S.

Chandrasekaran, “Flux: An adaptive partitioning operator for continuous query systems,” in Proc.

19th Int.

Conf.

Data Engineering (ICDE), 2003.

[15] S.

Chandrasekaran et al., “TelegraphCQ: Continuous dataflow processing for an uncertain world,” in Proc.

1st Biennial Conf.

Innovative Data Systems Research (CIDR), 2003.

[16] D.

J.

Abadi et al., “Aurora: A new model and architecture for data stream management,” The VLDB Journal, vol.

12, no.

2, pp.

120–139, 2003.

[17] A.

Arasu, S.

Babu, and J.

Widom, “The CQL continuous query language: Semantic foundations and query execution,” The VLDB Journal, vol.

15, pp.

121–142, 2006.

[18] M.

Stonebraker et al., “The 8 requirements of real-time stream processing,” ACM SIGMOD Record, vol.

34, no.

4, pp.

42–47, 2005.

[19] M.

Zaharia et al., “Discretized streams: An efficient and fault-tolerant model for stream processing on large clusters,” in Proc.

4th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud), 2012.

[20] M.

Budiu et al., “DBSP: Automatic incremental view maintenance for rich query languages,” The VLDB Journal, vol.

34, article 39, 2025.

[Online].

Available: https://doi.org/10.1007/s00778-025-00922-y

Detailed Matching

Matching passages are shown in the analysed-document pages using the numbered source markers. The table below preserves the corresponding source passage and match type.

No matching passages were found in the authorized source corpus.

Integrity Observations

Flags are evidence for review, not findings of misconduct.

No evidence-based integrity flags were detected by the configured checks.

Methodology and Limitations

Only meaningful sentences (at least 30 characters and 5 non-stopword tokens) are eligible. Exact matches count fully; near duplicates require at least 0.72 cosine similarity and contribute 0.6 of a sentence. Duplicate source hashes, duplicate candidate sentences, and overlapping candidate positions are counted once.

Source coverage is limited to the authorized uploaded-document corpus. Internet sources and indexed publications are unavailable unless configured. A similarity score is a review signal and is not, by itself, a conclusion of plagiarism. No-match results do not certify originality.